CampusFest Student Handbook

Oct 16, 2026 · @Kapil

Software Engineering in the Age of AI · MCA403-2 practical. Follow it step by step during the workshop; keep it as your revision guide for the eight laws and the syllabus terms behind them.

Before you start

You are **CampusFest Ltd.**, a software company. Your client wants "an app for managing college events". Over 150 minutes you will specify it, let an AI build it, design it for scale, test it, change it, audit it, sign it off and estimate it. Every stage breaks one of the eight laws from Part 1, on purpose.

Step 0 · Get an AI on a phone (first 15 minutes)

Your team needs **one** working AI, not one per person.

Try	Where	How to sign in
1 · Gemini	gemini.google.com or the Gemini app	Your Google account (no new password)
2 · ChatGPT	chatgpt.com or the app	"Continue with Google"
3 · Claude	claude.ai or the app	"Continue with Google"

1. Switch on mobile data if the Wi-Fi is slow or blocks sign-ups.
2. Sign in to the first tool that works.
3. Test it with this prompt, then raise your hand:

```
Write a Python function that adds two numbers.
```

No AI working? Tell the instructor. Your team will use a ready-made "AI-generated" app (App A, B or C) to work with, and every later stage still works.

Roles (rotate at every stage)

Role	Your job
Product Owner	Talks to stakeholders; owns the Story Card

Role	Your job
Architect	Owns the design and the ADR
Developer	Types the prompts; runs code at the station
QA	Reads the AI's assumptions aloud; tries to break everything

In a large team, two or three people share each role.

House rules

1. **Think before you prompt.** Paper first, phone second.
2. **Prove it.** The instructor may ask any of you to explain any line the AI produced. Can't explain it? Don't ship it: rewrite it, or ask the AI to explain until you can.
3. **Log it.** Every AI mistake goes in your team's "AI got it wrong" log (a sheet of paper or a shared note on a phone).
4. **Share the station.** Lab machines are shared run stations. Keep your turn short.

How each stage in this handbook works

Every stage has the same parts: **the law** (from Part 1), **the syllabus terms** you will use in exams, **what to do** step by step, **the prompts** to run, and **what you should notice**. Prompts are in boxes; copy them and replace the parts in angle brackets.

Stage 1 · Requirements

Law 1: Nobody knows what they want until they see it. (After Watts Humphrey's requirements uncertainty principle.) The hardest part of software is deciding exactly what to build. AI makes this matter more, not less: a vague request to a human gets questions back; a vague request to an AI gets a confident guess, built in seconds.

Remember from Part 1: the new ₹2,000 note (2016) was a different size, so about 2 lakh ATMs needed a technician visit before they could dispense it. "Must fit existing machines" was a hidden requirement. Aadhaar authentication, by contrast, answers one sharp question with Yes or No.

Syllabus terms (Units 1–2)

Term	Meaning	You do it when…
Requirements engineering	Discovering, writing, checking and managing what a system must do	All of Stage 1
Stakeholder	Anyone affected by the system	You meet the student, faculty, Dean and sponsor
Elicitation	Drawing requirements out of stakeholders by asking	The hot-seat interviews
Negotiation	Settling conflicts between stakeholders' wants	The Dean vs the sponsor
Validation	Checking the written requirement with stakeholders: "Is this what you meant?"	Reading your stories back
Functional requirement	What the system does	"Faculty can approve an event"
Non-functional requirement (NFR)	How well it does it: speed, security, privacy, scale	"Loads in under 2 s for 5,000 users"
User story	Who wants what, and why	Writing the Story Card
Acceptance criteria	Testable conditions that make a story "done"	Given / When / Then lines
Use case	Step-by-step interaction between an actor and the system	Your who-does-what diagram
Process model	The order in which you specify, build and test (Waterfall, Spiral, RAD, Scrum)	Working in short sprints today

What to do

1. **Read the brief** (7 minutes, paper only): "Build an app for managing college events." Write down what you would build.
2. **Compare** with other teams when asked. Notice how different the answers are.

3. **Interview stakeholders** (7 minutes). The instructor plays all four stakeholders in turn (a student, a faculty coordinator, the Dean and a sponsor). Your Product Owner asks questions; everyone takes notes. Ask open questions. Good questions reveal hidden constraints:
   - Who approves an event before it goes live?
   - What happens when an event is full?
   - Who sees students' contact details?
   - How many people will use it at the same time?
   - What does success look like for you?
4. **Write your Story Card** (13 minutes): 5 user stories, acceptance criteria for each, 2 measurable NFRs, and a MoSCoW priority (Must, Should, Could, Won't).

Story Card template

```
USER STORY
As a <who>, I want <what>, so that <why>.

ACCEPTANCE CRITERIA
Given <starting situation>
When <action>
Then <observable result, with a number if possible>

NON-FUNCTIONAL REQUIREMENTS (at least 2, each with a number)
- Performance: ______
- Security / privacy: ______

PRIORITY: Must / Should / Could / Won't
ASKED FOR BY: ______
```

Worked example

- **As a** student, **I want** to register for an event in one tap, **so that** I don't miss limited seats.
- **Given** the Hackathon has 100 seats and 99 are taken, **when** two students register at the same time, **then** exactly one gets the seat and the other is offered the waitlist.
- **NFR:** registration page loads in under 2 seconds with 5,000 students online.
- **NFR:** students' phone numbers are never shown to sponsors.

What you should notice

- "Fast" is not a requirement; "under 2 seconds" is. Numbers make requirements testable.

- Stakeholders disagree. Someone has to decide, and write the decision down.
- **Optional diagram:** open mermaid.live on a phone and draw who does what. A starter is in Stage 3.

Stage 2 · Let the AI build it

Law 1, again. An AI answers every question you forgot to ask, silently. The only defence is a precise specification: the chain **Requirement → User story → Acceptance criteria → Test case**. Every link you skip, the AI fills with a guess.

Syllabus terms

Term	Meaning
Specification	The precise, written statement of what to build, precise enough to test
Assumption	A decision made without being told; in AI output, usually invisible
Validation	Checking the product against what stakeholders actually wanted

What to do (15 minutes)

1. **Developer** opens the AI on the phone. Typing on a phone is slow, so start with your **two most important stories**.
2. Run **Prompt 2.1**. Do not skip lines 1 and 2 of the prompt: they force the AI to show its guesses.
3. **QA** reads the AI's assumptions aloud. Everyone else writes each one in the AI log.
4. Answer the AI's questions, then run **Prompt 2.2**.
5. You do not need to run the code. Read it and judge it.

Prompt 2.1 · Build from our stories

```
You are building a small Python app for our college.
Here are our user stories and acceptance criteria:
<paste or type your stories>

Before writing any code:
1. List every assumption you are making.
2. Ask me up to 3 clarifying questions.
Wait for my answers.
```

Prompt 2.2 · Build after answering

```
Here are my answers: <your answers>.
Now write the smallest working version in Python, in one file.
After the code, list which of our acceptance criteria it meets
and which it does not.
```

Score the AI's answer (5 minutes)

1. Read each **acceptance criterion** on your Story Card.
2. Find it in the AI's answer: **Yes**, it does it, or **No**. Count the Yes.
3. **Circle** anything the AI decided on its own that no stakeholder asked for.

Our criterion	AI did it?
Only 100 seats can be booked	Yes
Faculty approves before an event is public	No
Student gets a confirmation	Yes

Example score: **2 / 3**. AI decided alone: passwords stored as plain text.

Bonus, Prompt 2.3 · Fix the weakest criterion

```
This acceptance criterion was not met: <criterion>.
Rewrite only the code needed to meet it, and explain the change
in 3 lines.
```

What you should notice

- The AI rarely asks questions unless told to.
- Common silent assumptions: everyone is a student (no faculty role), no capacity limit, plain-text passwords, a date format nobody chose, nothing about privacy.

- The AI's own claim "this meets all criteria" is not evidence. Check each one yourself (house rule 2: **Prove it**).
- No AI? Open App A, B or C at your station (the instructor will tell you which) and use it as "your AI's output" and score it the same way.

Stage 3 · Design for 100,000 students

Law 2: Split the system by what will change. David Parnas (1972): hide each decision likely to change inside one module. Melvin Conway (1968): a system's structure ends up copying the structure of the team that built it.

Remember from Part 1: a UPI ID like ravi@okbank hides the bank account and IFSC; Ravi can change banks and nobody paying him notices. And Team B (one shared Notifier) changed 1 place when alerts moved from SMS to WhatsApp; Team A changed 14.

Syllabus terms (Unit 3)

Term	Meaning	In CampusFest
Abstraction	Use something through a simple idea, ignoring its insides	The screen calls `register()`; it doesn't know how seats are stored
Modularity	Divide the system into separate parts	Approval, Registration, Payment, Notifier
Information hiding	Each module hides a decision that may change	Only the Notifier knows it uses SMS
Separation of concerns	Each part handles one kind of job	Payment logic never sits in the UI
Functional independence	High cohesion (one job each), low coupling (few links)	Changing payment doesn't touch registration
Architecture	The big-picture structure: parts and how they connect	Your boxes-and-arrows sketch
Interface	The agreed way parts talk to each other	`register(student, event)`
Refactoring	Improving structure without changing behaviour	Turning one big file into modules

What to do (20 minutes)

1. **Imagine** results day: 100,000 students open the app at 9 am. Ask: what breaks first?
2. **Sketch by hand first** (5 minutes, no phone): boxes for parts, arrows for who calls whom.
3. **Run Prompt 3.1** and compare the AI's design with yours. Where did it add things (cache, queue, load balancer)? Can your Architect explain each one? If not, ask the AI with Prompt 3.2.
4. **Write one ADR** (Architecture Decision Record) for your most important decision.

Prompt 3.1 · Ask for an architecture

```
Our college event app must handle 100,000 students logging in
within 10 minutes on results day. Features: <list your Must stories>.
Propose a simple architecture. List the components, what each one
does, and how they talk to each other. Then draw it as a Mermaid
flowchart.
```

Prompt 3.2 · Make it explain itself

```
Explain <component> as if to a first-year student. What problem does
it solve here, and what would happen without it?
```

Diagram starter (paste into mermaid.live)

```
flowchart LR
  Student -->|register| UI
  UI --> Registration
  Registration --> DB[(Database)]
  Faculty -->|approve| Approval
  Approval --> DB
  Registration --> Notifier
```

ADR template

```
ADR-001: <title, e.g. "Separate Registration from the UI">
CONTEXT: What problem forced this decision?
OPTIONS: A) ____  B) ____  C) what the AI suggested: ____
DECISION: We chose ____ because ____
CONSEQUENCES: Easier: ____   Harder: ____
PRINCIPLE: modularity / information hiding / separation of concerns
```

What you should notice

- An AI asked for one screen at a time usually produces Team A's design: each prompt is local, and the AI doesn't know what will change.
- The AI can propose; only a human can own the trade-off. That's why the DECISION line in the ADR is written by you.

Stage 4 · The Last Seat and testing

Law 5: Works for one user, breaks for a million. Demos have one user; production has thousands arriving at the same second. **Law 4: Testing shows bugs are present, never that they are absent** (after Edsger Dijkstra). You can't test every input, so you choose the inputs most likely to break things.

Remember from Part 1: Flipkart's Big Billion Day (2014) accepted orders for stock that had already sold. Thousands chase the last Tatkal berth in the same second. Chandrayaan-3 succeeded after ISRO switched to "failure-based design": first ask what can fail.

The Last Seat, explained

The Hackathon has 100 seats; 99 are taken. 50 students click Register at the same instant. The AI's code does two steps:

1. **Check:** is a seat free? (Yes, 1 left.)
2. **Act:** add the student.

Between check and act there is a tiny gap (a database round-trip). All 50 students check during that gap, all 50 see "1 left", and all 50 are added. The demo run put **149 people in 100 seats**, with no error message. This is a **race condition**.

The fix: make check-and-act one indivisible (atomic) step: a lock in the code, or a transaction or constraint in the database. In the code kit, compare `NaiveEvent` with `SafeEvent` in `race_demo/last_seat.py`.

Syllabus terms (Unit 4)

Term	Meaning	Example
Black-box testing	Tests designed from the requirement: inputs and expected outputs	Age 18 must be accepted
White-box testing	Tests designed from the code's structure and paths	The gap between check and act

Term	Meaning	Example
Boundary value analysis	Test at the edges of allowed ranges	99 → 100 seats; 100 → 101
Unit testing	Test one function or class alone	`register()` by itself
Integration testing	Test parts working together	`register()` with the real database
System testing	Test the whole app as a user would	Student books, gets confirmation
Load / stress testing	Test at and beyond the expected peak	50 clicks at once; 100,000 logins
Verification vs validation	"Did we build it right?" vs "Did we build the right thing?"	Bug vs wrong requirement

What to do (10 minutes)

1. **On paper first**, fill the Test Design Sheet for your app (or for the Hackathon example):

#	Class	Example	Expected
1	Normal	10 taken, Asha registers	Accepted
2	Boundary	99 taken, Ravi registers	Accepted; exactly 100
3	Edge / negative	100 taken, Meera registers	Rejected; offered waitlist
4	Concurrency	99 taken, 50 click together	Exactly 1 accepted
5	Failure	Database is down	??? (write the missing requirement)

2. **Then** run Prompt 4.1 and tick which of your five classes the AI covered.
3. **Optional, at your station:** run `python test_last_seat.py` in the `race_demo` folder to see all five classes as runnable code.

Prompt 4.1 · Ask for tests

```
Write tests in Python (unittest) for this registration function:
<paste the function>
The event has a capacity of 100.
```

Prompt 4.2 · Push for the hard cases

```
Which of these did your tests miss: boundary values, concurrent
registrations for the last seat, and the database being unavailable?
Add one test for each, and explain what each one proves.
```

What you should notice

- AI usually tests the cases it coded for (Normal, maybe Edge). Concurrency and Failure are usually missing.
- Row 5 has no expected result because no requirement says what should happen. **Testing finds missing requirements, not just bugs.** Add it to your Story Card.
- Code that "runs without errors" can still be wrong. 149 / 100 ran perfectly.

Stage 5 · The change request

Law 6: Software that is used must keep changing (Meir Lehman, 1974), or it becomes steadily less useful. And every change is a risk.

Remember from Part 1: Y2K forced the world to rewrite old code before a deadline that couldn't move, and much of that work came to India. On 19 July 2024, one faulty CrowdStrike update sent to everyone at once crashed millions of Windows machines; Indian airlines handed out handwritten boarding passes. Ariane 5 (1996) reused code that had been proven on Ariane 4, without re-testing it, and self-destructed 37 seconds after launch.

Syllabus terms (Units 1, 4, 5)

Term	Meaning
Maintenance & evolution	Changing software after release; most of its lifetime cost
Corrective / adaptive / perfective / preventive maintenance	Fix bugs / adapt to a new environment / add features / restructure to avoid future problems
Regression testing	Re-running old tests after a change to prove nothing that worked is now broken
DevOps, CI/CD	Every change goes through an automated pipeline: build → test → deploy → monitor → feedback
Staged rollout	Release to a few users first, watch, then widen

What to do (10 minutes)

The Dean has called: **"Add a waitlist. When someone cancels, the first person on the waitlist gets the seat."**

1. Before prompting, write down which of your five tests from Stage 4 might be affected.
2. Run Prompt 5.1.
3. Re-check all five tests against the new code (on paper, or by running them at your station).
4. Record: which tests still pass, which broke, and why.

Prompt 5.1 · Make the change

```
Add a waitlist to this registration code: when an event is full,
new students join a waitlist; when someone cancels, the first person
on the waitlist gets the seat.
<paste the current code>
Change as little as possible, and list every function you changed.
```

Prompt 5.2 · Check what might break

```
Here are our existing tests: <paste or describe them>.
Which of them could fail because of the waitlist change, and why?
```

What you should notice

- A "simple" change often breaks a test that passed an hour ago, usually the boundary or edge test, because "full" now means something different.
- Some AIs rewrite the whole function and quietly bring the race condition back.
- The only way to know nothing else broke is to re-run every test. Automating that on every change is the heart of DevOps.

Stage 6 · Release audit and sign-off

Law 7: You ship everything you import. Libraries, AI-suggested packages, even what your code writes into logs: all of it is your responsibility. **Law 8: You can delegate the work, never the responsibility.** An AI can write the app; a human signs the release.

Remember from Part 1: AIIMS Delhi (November 2022) ran outpatient care on paper for over a week after a ransomware attack; the backups that survived were the ones disconnected from the network. Log4Shell (2021) was one free logging library hidden inside millions of apps. AI models invent package names, and attackers register them ("slopsquatting"). When the 2021 income-tax portal failed, the Finance Ministry summoned the vendor's CEO. An AI agent deleted a live database during a code freeze in 2025.

Syllabus terms (Unit 4, CO4)

Term	Meaning
Software quality	Correctness, reliability, security, privacy, performance, maintainability, usability
Quality assurance & reviews	Systematically checking work, including code you didn't write
Validation / acceptance	Confirming with stakeholders that the product meets their requirements before release
Least privilege	Every person or agent gets only the access it needs
Accountability	A named human answers for the release

What to do (15 minutes, at your station)

1. Open your assigned app in the `fallback_apps` folder (or audit your own AI's app) and run it: `python app_a_eventhub.py` (or `app_b_festflow.py`, `app_c_campuspass.py`).

2. Read the code aloud as a team. Use the checklist and try to find one of each:

Find one	Ask yourself
Functional defect	Does it do what the acceptance criteria say? Try the boundary values.
Security issue	Can I see or change what I shouldn't? Try a strange input.
Missing requirement	Which stakeholder's need is ignored?
Performance concern	What loops over everything, every time? What happens at 100,000 users?
Bad AI assumption	Who decided this? Where is it written?
Privacy leak	Is personal data shown, logged or kept that shouldn't be? (India's DPDP Act, 2023)

3. For each finding, write the test that would have caught it.
4. **Sign off.** Every team member signs one line:

```
We, the undersigned, WOULD / WOULD NOT release this software.
Reason: _______________________________
Signed: ______  ______  ______  ______
```

Prompt 6.1 · Use the AI as a second reviewer (after your own review)

```
Review this code as a strict security and quality reviewer. List the
top 5 problems in order of severity, with the line where each occurs
and how to fix it.
<paste code>
```

Before you install anything an AI suggests, check five things: it exists on the official registry (pypi.org, npmjs.com); the spelling is exact; you know who maintains it and since when; it has a real source repository; and you actually need it.

What you should notice

- Your own review should come first. If you ask the AI first, you stop thinking (Part 1: AI users learned less when they fully delegated).
- The AI reviewer will find some problems and miss others. Compare its list with yours.
- Your signature is the point of this stage. If you can't explain it, you don't ship it.

Stage 7 · Estimation

Law 3: Adding people to a late project makes it later (Fred Brooks, *The Mythical Man-Month*, 1975). New people need teaching, and communication paths grow as n(n−1)/2: 4 people = 6 paths, 10 people = 45. And Hofstadter's law: it always takes longer than you expect, even when you account for Hofstadter's law.

Remember from Part 1: "login with OTP" looked like two screens, but hid OTP expiry, resend limits, wrong numbers, DND numbers, rate limiting, logs, tests and monitoring. In METR's 2025 trial, experienced developers *felt* faster with AI but were measured 19% *slower*. Feelings are not data.

Syllabus terms (Unit 5)

Term	Meaning
Software scope	Everything the work really includes, visible or not
Decomposition	Break the work into parts, estimate each, add them up
Empirical estimation model	A formula fitted on data from past projects, e.g. COCOMO II
Person-month (PM)	One person working for one month
Process & project metrics	Measuring what actually happened, to improve the next estimate

COCOMO II in one formula

```
\text{Effort (PM)} = 2.94 \times \text{Size}^{E} \times \prod EM_i
```

- **Size:** thousands of lines of code (KSLOC).
- **E:** an exponent of about 1.10 for a typical project (0.91 plus five scale factors). Because it is above 1, double the size means *more* than double the effort.
- **EM:** effort multipliers, 1.00 when normal. Higher reliability needs push effort up (1.10); a less experienced team pushes it up (1.15); strong tools pull it down (0.78).

Worked example (CampusFest, 3 KSLOC): 2.94 × 3^1.10 × 1.085 ≈ **10.7 person-months**, about **7.8 months** of schedule.

What to do (5 minutes)

1. **Gut:** agree a team estimate for the full app (all four stakeholders happy). 30 seconds.

2. **AI:** run Prompt 7.1.
3. **Model:** your instructor runs the COCOMO II calculator (`python cocomo2_quick.py` in the `estimation` folder). Try your own size at a station: `python cocomo2_quick.py 5`.
4. Fill in the table:

Estimate	Full app (person-months)	How we got it
Our gut		
Our AI		
COCOMO II	10.7	3 KSLOC, calculator

Prompt 7.1 · Ask for an estimate, with its working

```
Estimate how long a team of first-year MCA students would take to
build this app: <list your Must stories and NFRs>.
Break the work into tasks, estimate each in hours, and add them up.
State your assumptions.
```

What you should notice

- Gut estimates are usually far lower than the model: we only see the visible work.
- The calculator assumes AI tools cut effort by about 22%. METR measured experienced developers 19% slower. Which is true for your team? You only find out by measuring.
- Prompt 7.1 asks for decomposition on purpose: the task list is more useful than the final number.

Reference for revision

The eight laws at a glance

#	Law	Story to remember	Syllabus link
1	Nobody knows what they want until they see it	₹2,000 note and 2 lakh ATMs; Aadhaar's Yes/No	Requirements engineering, process models (Units 1–2)
2	Split the system by what will change	UPI ID hides your bank account	Abstraction, modularity, information hiding (Unit 3)
3	Adding people to a late project makes it later	Hidden work in "login with OTP"; METR 19%	Estimation, COCOMO II, metrics (Unit 5)

#	Law	Story to remember	Syllabus link
4	Testing shows bugs, never their absence	Chandrayaan-3's failure-based design	Black/white-box, unit to system testing (Unit 4)
5	Works for one user, breaks for a million	Flipkart BBD 2014; Tatkal; Hotstar 25.3M	Performance NFRs, concurrency, load testing
6	Software that is used must keep changing	Y2K; CrowdStrike boarding passes	Maintenance, regression testing, DevOps
7	You ship everything you import	AIIMS ransomware; Log4Shell	Software quality, security, reviews (Unit 4)
8	You can delegate the work, never the responsibility	Income-tax portal 2021; AI agent deletes database	Validation, quality management, CO4

Five habits of an AI-era engineer

1. **Think first:** paper before prompt.
2. **Specify:** write it so a machine can't guess wrong.
3. **Verify:** test what can fail, not what works.
4. **Measure:** trust numbers over feelings.
5. **Own it:** if you can't explain it, don't ship it.

Prompt cheat sheet

Goal	Add this to your prompt
Surface hidden guesses	"Before writing code, list every assumption you are making."
Stop it guessing	"Ask me up to 3 clarifying questions first, and wait."
Keep changes small	"Change as little as possible and list every function you changed."
Check against requirements	"List which acceptance criteria this meets and which it does not."
Get hard tests	"Include boundary, concurrency and failure tests."
Understand it yourself	"Explain this as if to a first-year student."

Goal	Add this to your prompt
Use AI as reviewer	"Review this as a strict security reviewer; list the top 5 problems with line numbers."

Safety rules for AI help: never paste passwords, API keys or anyone's personal data into an AI tool; check every package an AI suggests before installing it; and never give an AI agent access that can delete real data without a human approving it.

What your team submits (CIA 1)

- [ ] Story Cards (Stage 1)
- [ ] AI score sheet and AI-got-it-wrong log (Stage 2)
- [ ] Architecture sketch and ADR (Stage 3)
- [ ] Test Design Sheet (Stage 4)
- [ ] Change-request notes: which tests broke and why (Stage 5)
- [ ] Signed release audit (Stage 6)
- [ ] Three-estimate table (Stage 7)

Further reading

- Roger Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. (your textbook)
- Ian Sommerville, *Software Engineering*
- Fred Brooks, *The Mythical Man-Month* and the essay "No Silver Bullet"
- David Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules" (1972)
- NPTEL Software Engineering course: onlinecourses.nptel.ac.in/noc20_cs68